

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf\_d9866476-800\agent-a1ec2f355d490cd7b.jsonl`
- SHA-256: `e96aa498065704161493f5e1a028fcedc967d39218c9c7d7e59149810b1a84f7`
- Source modified: `2026-06-22T09:11:19+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf\_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`

Transcript

1. user (2026-06-22T09:08:03.305Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

CONFIGURED + CHECKPOINTED STATE: define exactly what running "as configured and checkpointed" means. Where does the WASB model checkpoint/weights live (backend/pipeline/detectors/, config paths, download)? How is config.json resolved (backend/config/loader.py, defaults in models.py, RALLY_APP_HOME/app-data home)? How to PIN a fixed config for a reproducible nightly run (so a config-default change is a DELIBERATE baseline refresh, not silent drift). Give the exact run recipe to process a video with the default-configured segmenter + checkpoint headlessly.

2. assistant (2026-06-22T09:08:05.653Z)

I'll map the badminton-highlight-indexer repo to understand the configured/checkpointed pipeline and design a nightly regression test. Let me start by exploring the structure.

3. user (2026-06-22T09:08:06.489Z)

```
.claude\worktrees\nightly-e2e\backend\annotations\__init__.py
.claude\worktrees\nightly-e2e\backend\annotations\automated_provider.py
.claude\worktrees\nightly-e2e\backend\annotations\base.py
.claude\worktrees\nightly-e2e\backend\annotations\file_provider.py
.claude\worktrees\nightly-e2e\backend\api\__init__.py
.claude\worktrees\nightly-e2e\backend\api\auth.py
.claude\worktrees\nightly-e2e\backend\api\job_worker.py
.claude\worktrees\nightly-e2e\backend\api\uploads.py
.claude\worktrees\nightly-e2e\backend\billing.py
.claude\worktrees\nightly-e2e\backend\compute\__init__.py
.claude\worktrees\nightly-e2e\backend\compute\base.py
.claude\worktrees\nightly-e2e\backend\compute\cloud_run.py
.claude\worktrees\nightly-e2e\backend\config\__init__.py
.claude\worktrees\nightly-e2e\backend\config\loader.py
.claude\worktrees\nightly-e2e\backend\config\models.py
.claude\worktrees\nightly-e2e\backend\config\presets.py
.claude\worktrees\nightly-e2e\backend\config\startup.py
.claude\worktrees\nightly-e2e\backend\eval\__init__.py
.claude\worktrees\nightly-e2e\backend\eval\ablation.py
.claude\worktrees\nightly-e2e\backend\eval\ablation_pdf.py
.claude\worktrees\nightly-e2e\backend\eval\annotations.py
.claude\worktrees\nightly-e2e\backend\eval\audio\__init__.py
.claude\worktrees\nightly-e2e\backend\eval\audio\e1_probe.py
```

.claude\worktrees\nightly-e2e\backend\eval\batch.py
.claude\worktrees\nightly-e2e\backend\eval\calibrate_local.py
.claude\worktrees\nightly-e2e\backend\eval\calibrate_wasb.py
.claude\worktrees\nightly-e2e\backend\eval\calibration.py
.claude\worktrees\nightly-e2e\backend\eval\classifier.py
.claude\worktrees\nightly-e2e\backend\eval\distill_local.py
.claude\worktrees\nightly-e2e\backend\eval\eval_partial_labels.py
.claude\worktrees\nightly-e2e\backend\eval\eval_stats.py
.claude\worktrees\nightly-e2e\backend\eval\experiment.py
.claude\worktrees\nightly-e2e\backend\eval\export_wasb_segments.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_audio.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_compare.py
.claude\worktrees\nightly-e2e\backend\eval\fusion_golden.py
.claude\worktrees\nightly-e2e\backend\eval\gemini_refine.py
.claude\worktrees\nightly-e2e\backend\eval\golden_fixtures.py
.claude\worktrees\nightly-e2e\backend\eval\golden_real_fixtures.py
.claude\worktrees\nightly-e2e\backend\eval\gt_loader.py
.claude\worktrees\nightly-e2e\backend\eval\harness.py
.claude\worktrees\nightly-e2e\backend\eval\metrics.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_eval.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_gate.py
.claude\worktrees\nightly-e2e\backend\eval\per_user_regression.py
.claude\worktrees\nightly-e2e\backend\eval\promotion.py
.claude\worktrees\nightly-e2e\backend\eval\rally_seg_eval.py
.claude\worktrees\nightly-e2e\backend\eval\rally_seq_proto.py
.claude\worktrees\nightly-e2e\backend\eval\regression.py
.claude\worktrees\nightly-e2e\backend\eval\score_calibration.py
.claude\worktrees\nightly-e2e\backend\eval\segmentation_metrics.py
.claude\worktrees\nightly-e2e\backend\eval\served_gate_a.py
.claude\worktrees\nightly-e2e\backend\eval\serve_contrast.py
.claude\worktrees\nightly-e2e\backend\eval_splits.py
.claude\worktrees\nightly-e2e\backend\eval\tolerance_metrics.py
.claude\worktrees\nightly-e2e\backend\eval\training.py
.claude\worktrees\nightly-e2e\backend\eval>windowing.py
.claude\worktrees\nightly-e2e\backend\features__init__.py
.claude\worktrees\nightly-e2e\backend\features\rally_seq.py
.claude\worktrees\nightly-e2e\backend\infrastructure\database.py
.claude\worktrees\nightly-e2e\backend\infrastructure\execution_policy.py
.claude\worktrees\nightly-e2e\backend\main.py
.claude\worktrees\nightly-e2e\backend\pipeline\audio__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\audio\hit_detector.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\audio_features.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\court_gate.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\device.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\fusion_features.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\native_wasb_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\person_detector.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\rally_gate.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\stub_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\nightly-e2e\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\nightly-e2e\backend\pipeline\personalization__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\personalization\serving.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters_keyhint.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\fusion_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\gemini.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\motion.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\tracknet_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\trajectory_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\wasb_hybrid.py

```
.claude\worktrees\nightly-e2e\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\compiler.py
.claude\worktrees\nightly-e2e\backend\pipeline\stitcher\watermark_i18n.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\__init__.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\base.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\blur.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\format.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\pts_continuity.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\relevance.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\resolution_fps.py
.claude\worktrees\nightly-e2e\backend\pipeline\validators\scene_cut.py
.claude\worktrees\nightly-e2e\backend\providers\base.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-06-22T09:08:08.274Z)

```
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/.github/workflows/ci.yml
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/reporting/spec.yaml
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/ui/ui_version.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/config.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/experiment_leaderboard.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/clean_single_rally/expected.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/clean_single_rally/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/manifest.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-
e2e/eval_baselines/fixtures/multi_rally_with_deadtime/expected.json
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-
e2e/eval_baselines/fixtures/multi_rally_with_deadtime/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/occlusion_break/expected.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/fixtures/occlusion_break/provenance.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/gemini_wrapper_2026-06-10.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/heuristic_lovo_n6.json
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/eval_baselines/nightly_baseline.json
```

5. user (2026-06-22T09:08:09.082Z)

```
total 207
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609  611 Jun 22 14:36 .coveragerc
-rw-r--r-- 1 avidu 197609   87 Jun 22 14:36 .git
-rw-r--r-- 1 avidu 197609  947 Jun 22 14:36 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .github
-rw-r--r-- 1 avidu 197609 1117 Jun 22 14:36 .gitignore
-rw-r--r-- 1 avidu 197609  525 Jun 22 14:36 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35347 Jun 22 14:36 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 backend
-rw-r--r-- 1 avidu 197609 2735 Jun 22 14:36 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 deploy
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 docs
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 eval_baselines
-rw-r--r-- 1 avidu 197609  234 Jun 22 14:36 mypy.ini
-rw-r--r-- 1 avidu 197609  452 Jun 22 14:36 pyproject.toml
```

```

-rw-r--r-- 1 avidu 197609 245 Jun 22 14:36 pytest.ini
-rw-r--r-- 1 avidu 197609 642 Jun 22 14:36 requirements.txt
-rw-r--r-- 1 avidu 197609 884 Jun 22 14:36 ruff.toml
-rwxr-xr-x 1 avidu 197609 2055 Jun 22 14:36 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5620 Jun 22 14:36 run_eval.py
-rw-r--r-- 1 avidu 197609 4356 Jun 22 14:36 run_regression.py
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 scratch
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 scripts
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 tests
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 tools
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 training

```

6. assistant (2026-06-22T09:08:11.037Z)

Now let me examine the key configuration and pipeline files:

7. user (2026-06-22T09:08:12.385Z)

```

1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,
15                     40
16                 ],
17                 "court_green_upper": [
18                     85,
19                     255,
20                     255
21                 ],
22                 "court_pixels_min_ratio": 0.05
23             }
24         },
25         "indexing": {
26             "default_segmenter": "gemini",
27             "use_gpu": true,
28             "motion_threshold": 0.08,
29             "min_segment_duration": 3.0,
30             "dead_time_merge_gap": 2.0,
31             "chunk_duration": 90,
32             "chunk_overlap": 10,
33             "detector_model": "fasterrcnn_resnet50_fpn",
34             "detector_score_threshold": 0.5,
35             "yolo_velocity_threshold": 0.15,
36             "yolo_frame_skip": 3,
37             "yolo_tracker": "bytetrack.yaml",
38             "yolo_prefetch_workers": 2,
39             "min_action_window_duration": 2.0,
40             "log_yolo_candidates": true,
41             "store_yolo_candidates": true,
42             "ai_handoff_padding": 2.0,
43             "ai_max_workers": 1,
44             "tracknet": {
45                 "wsl_distro": "Ubuntu",
46                 "repo_dir": "~/models/TrackNetV4",
47                 "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",

```

```

48         "conda_env": "TrackNetV4",
49         "weights_path": "",
50         "queue_length": 5,
51         "stage_in_wsl": true,
52         "wsl_stage_dir": "~/clips",
53         "timeout_sec": 1800,
54         "velocity_threshold": 0.02
55     },
56     "wasb": {
57         "wsl_distro": "Ubuntu",
58         "repo_dir": "~/models/WASB-SBDT",
59         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62         "sport": "badminton",
63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68 "output": {
69     "default_highlights_name": "highlights.mp4",
70     "db_name": "sports_indexer.db"
71 },
72 "stitching": {
73     "begin_padding": 0.5,
74     "end_padding": 0.5,
75     "use_cache": false,
76     "min_shot_count": 1,
77     "watermark": {
78         "enabled": true,
79         "text": "Generated by Khelsutra.guru",
80         "position": "bottom-right",
81         "opacity": 0.85,
82         "logo_path": "",
83         "font_path": ""
84     }
85 }
86 }

```

8. user (2026-06-22T09:08:13.113Z)

```

1     """Configuration loader and saver.
2
3     Provides a single place to load validated AppConfig with built-in defaults
4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     import os
14     from pathlib import Path
15     from typing import Any
16
17     from pydantic import BaseModel
18
19     from backend.utils.app_home import default_config_path
20
21     from .models import AppConfig
22     from .presets import (

```

```

23     COMPUTE_TARGET_FOR_PROFILE,
24     PROFILE_PRESETS,
25     is_known_profile,
26     preset_for,
27     suggest_profile,
28 )
29
30 logger = logging.getLogger(__name__)
31
32 # Config path resolution now lives in ``backend.utils.app_home.default_config_path()`` ?
it honours
33 # ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and equals ``Path("config.json")`` (CWD-
relative) when
34 # the env is unset, exactly the pre-P0a value. (The old module-level
``DEFAULT_CONFIG_PATH`` constant
35 # was dropped: it had no external consumers and a frozen-at-import snapshot would
silently diverge
36 # from a later RALLY_APP_HOME.)
37
38
39 def _deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
40     """Recursively overlay ``override`` onto ``base`` (override wins; dicts merge,
scalars
41     replace). Used ONLY for the profile-preset precedence path so a config.json sub-key
42     overrides a preset sub-key while sibling preset/default sub-keys survive ? the no-
profile
43     path keeps the original shallow ``{**defaults, **file_data}`` merge unchanged."""
44     out = dict(base)
45     for key, val in override.items():
46         if isinstance(val, dict) and isinstance(out.get(key), dict):
47             out[key] = _deep_merge(out[key], val)
48         else:
49             out[key] = val
50     return out
51
52
53 def _resolve_explicit_profile(file_data: dict[str, Any]) -> str:
54     """The EXPLICITLY chosen deployment profile, env winning over config.json. ``""`` if
none
55     is set. Auto-detect is deliberately NOT consulted here ? it only suggests (D15), so
a bare
56     environment never silently selects a profile (and never silently spends on cloud).
57
58     An *unrecognised* ``DEPLOYMENT_PROFILE`` env value warns and FALLS THROUGH to the
file's
59     profile (rather than suppressing it) ? so an env typo can't leave the recorded
profile
60     asserting a preset that was never applied. A bad value in config.json is left to
surface as
61     a hard, typed-Literal error downstream (config-file correctness)."""
62     env_profile = os.environ.get("DEPLOYMENT_PROFILE")
63     if env_profile and env_profile.strip():
64         ep = env_profile.strip()
65         if is_known_profile(ep):
66             return ep
67         logger.warning(
68             "[CONFIG] unrecognized DEPLOYMENT_PROFILE env value %r (valid: %s); ignoring
it "
69             "and falling back to config.json deployment.profile.", ep, ",
".join(PROFILE_PRESETS),
70         )
71         # fall through to the file's profile (env typo must not suppress a valid file
choice)
72     dep = file_data.get("deployment")
73     if isinstance(dep, dict):

```

```

74         prof = dep.get("profile")
75         if isinstance(prof, str) and prof.strip():
76             return prof.strip()
77     return ""
78
79
80     def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
81     "") -> list[str]:
82         """Dotted paths in `data` that the typed config will not honor.
83
84         Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
85         kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
86         typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
87         user's intent is lost ? collect every such key so load_config can warn.
88         """
89         unknown: list[str] = []
90         fields = model_cls.model_fields
91         for key, val in data.items():
92             if key not in fields:
93                 unknown.append(prefix + key)
94                 continue
95             ann = fields[key].annotation
96             if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
97             BaseModel):
98                 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
99         return unknown
100
101     def _get_builtin_defaults() -> dict[str, Any]:
102         """Return a plain dict of the current built-in defaults.
103
104         The AppConfig model is the single source of truth for defaults.
105         """
106         return AppConfig().model_dump(mode="json")
107
108     def load_config(path: str | Path | None = None) -> AppConfig:
109         """Load configuration with the following precedence:
110
111         1. Built-in defaults defined in the Pydantic models.
112         2. Values from the JSON file (if present and readable).
113         3. (Future) environment / CLI overrides.
114
115         Missing file or unreadable file ? returns a fully defaulted AppConfig.
116         Unknown keys in the file are tolerated (extra="allow" on the model).
117         """
118         cfg_path = Path(path) if path is not None else default_config_path()
119
120         file_data: dict[str, Any] = {}
121         if cfg_path.exists():
122             try:
123                 with open(cfg_path, "r", encoding="utf-8") as f:
124                     file_data = json.load(f)
125             except Exception as exc:
126                 # Non-fatal: continue with defaults + whatever we could parse.
127                 # In production we might log here.
128                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
129                 data.")
130
131         # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`,
132         # `null`) would
133         # otherwise crash startup: a truthy non-mapping hits `.items()` in
134         _unknown_key_paths, and any
135         # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
136         warn, per the

```

```

133     # loader's "bad input is non-fatal" contract.
134     if not isinstance(file_data, dict):
135         logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
136                        cfg_path, type(file_data).__name__)
137         file_data = {}
138
139     if file_data:
140         unknown = _unknown_key_paths(file_data, AppConfig)
141         if unknown:
142             logger.warning(
143                 "[CONFIG WARNING] %s contains keys the typed config does not "
144                 "recognize (typo'd or removed knobs ? top-level extras are kept "
145                 "but unread; unknown section keys are silently dropped): %s",
146                 cfg_path, ", ".join(sorted(unknown)),
147             )
148
149     # Precedence: built-in defaults < profile preset < config.json (< env/--set, applied
at
150     # consumption sites downstream). A *deployment profile* is opt-in ? it is applied
ONLY when
151     # explicitly selected (DEPLOYMENT_PROFILE env or config.json deployment.profile).
With no
152     # profile the merge is byte-identical to the pre-M1 loader, so the default path is
unchanged.
153     defaults = _get_builtin_defaults()
154     profile = _resolve_explicit_profile(file_data)
155
156     if profile and not is_known_profile(profile):
157         # Only an unrecognised value from config.json reaches here (env typos already
fell back
158         # in _resolve_explicit_profile). Warn + apply no preset; the bad value also hits
the
159         # typed Literal at model_validate below, a hard clear error ? loud, never
silent.
160         logger.warning(
161             "[CONFIG] unrecognized deployment profile %r (valid: %s); applying no
preset.",
162             profile, ", ".join(PROFILE_PRESETS),
163         )
164         profile = ""
165
166     if profile:
167         merged = _deep_merge(_deep_merge(defaults, preset_for(profile)), file_data)
168         # Record the profile actually applied ? the env may have chosen it over
config.json,
169         # so re-stamp it onto the merged deployment section to keep the record honest.
170         merged.setdefault("deployment", {})
171         if isinstance(merged["deployment"], dict):
172             merged["deployment"]["profile"] = profile
173             ct = merged["deployment"].get("compute_target")
174             canonical = COMPUTE_TARGET_FOR_PROFILE.get(profile)
175             if not ct and canonical:
176                 # An active profile with no explicit compute_target (e.g. config.json
blanked it
177                 # to "") gets its canonical target back ? the record never silently lies
about an
178                 # active profile's compute_target (it stays meaningful for the M2+
ComputeBackend).
179                 merged["deployment"]["compute_target"] = canonical
180                 elif ct and canonical and ct != canonical:
181                     logger.warning(
182                         "[CONFIG] deployment profile %r normally uses compute_target=%r but
"
183                         "config.json overrides it to %r ? honoring the explicit override.",

```



```

184         profile, canonical, ct,
185     )
186     logger.info("[CONFIG] deployment profile ACTIVE: %s (compute_target=%s)",
187                profile, merged["deployment"].get("compute_target"))
188 else:
189     # No profile ? the original shallow merge (Pydantic re-defaults nested
sections).
190     merged = {**defaults, **file_data}
191     suggestion = suggest_profile() # cheap, env-only (no torch); SUGGESTS only
(D15)
192     if suggestion:
193         logger.debug(
194             "[CONFIG] no deployment.profile set (manual knobs). Environment suggests
%r ? "
195             "set deployment.profile or DEPLOYMENT_PROFILE to apply (auto-detect
never "
196             "auto-applies ? D15).", suggestion,
197         )
198
199     return AppConfig.model_validate(merged)
200
201
202 def save_default_config(path: str | Path | None = None) -> Path:
203     """Write a fresh default config.json.
204
205     Used by scripts/doctor.py diagnostics and the --setup CLI path.
206     Overwrites if the file exists.
207     """
208     cfg_path = Path(path) if path is not None else default_config_path()
209     cfg = AppConfig()
210     cfg_path.parent.mkdir(parents=True, exist_ok=True)
211     cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
212     return cfg_path
213
214
215     # The shipped batteries-included LIGHT-tier default (PACKAGING_PLAN.md P2b / gate 08).
216     # Deliberately MINIMAL: the `truly-local` PROFILE preset (presets.py) supplies
skip_ai_handoff,
217     # local storage and the compute_target, so this stays the handful of knobs a fresh, non-
technical
218     # user gets ? a torch-free, offline, zero-setup experience:
219     # * default_segmenter "motion" ? pure-OpenCV, no torch / GPU / weights / API key
(LIGHT tier);
220     # * profile "truly-local" ? no Gemini handoff, local storage ? never surprise-
bills on
221     # Gemini nor returns 0 rallies on WASB-without-a-key
(08 intent);
222     # * use_gpu false ? honest for a LIGHT (no-GPU) box (motion ignores it
either way).
223     # The first-run wizard (P2c) upgrades this to Gemini once the user supplies a key.
224     LIGHT_DEFAULT_CONFIG: dict[str, Any] = {
225         "sport": "badminton",
226         "deployment": {"profile": "truly-local"},
227         "indexing": {"default_segmenter": "motion", "use_gpu": False},
228     }
229
230
231 def write_light_default_config(
232     path: str | Path | None = None, *, overwrite: bool = False
233 ) -> tuple[Path, bool]:
234     """Seed the batteries-included LIGHT default config (`LIGHT_DEFAULT_CONFIG`).
235
236     Returns `(path, written)`. Unlike :func:`save_default_config` (which dumps a FULL
AppConfig
237     and overwrites), this writes the minimal profile-driven LIGHT bundle and ? by

```

```

default ? **never
238     overwrites an existing file**, so a user's edits or the first-run wizard's choices
are preserved
239     (idempotent first-run seeding). Pass ``overwrite=True`` to force. Creates parent
dirs.
240     """
241     cfg_path = Path(path) if path is not None else default_config_path()
242     if cfg_path.exists() and not overwrite:
243         return cfg_path, False
244     cfg_path.parent.mkdir(parents=True, exist_ok=True)
245     cfg_path.write_text(json.dumps(LIGHT_DEFAULT_CONFIG, indent=2) + "\n",
encoding="utf-8")
246     return cfg_path, True
247
248
249     # Backwards-compatible alias used by some older call sites during transition.
250     def get_default_config() -> dict[str, Any]:
251         """Return defaults as a plain dict (for transitional use only)."""
252         return _get_builtin_defaults()
253

```

9. user (2026-06-22T09:08:14.213Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33     class TrackNetConfig(BaseModel):
34         wsl_distro: str = "Ubuntu"
35         repo_dir: str = "~/models/TrackNetV4"
36         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37         conda_env: str = "TrackNetV4"
38         weights_path: str = ""
39         queue_length: int = 5
40         stage_in_wsl: bool = True
41         wsl_stage_dir: str = "~/clips"

```

```

42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54     class WasbIndexConfig(BaseModel):
55         """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58         these knobs actually take effect ? previously this whole block was silently dropped
59         because nested config sections defaulted to extra='ignore'.
60         """
61         wsl_distro: str = "Ubuntu"
62         repo_dir: str = "~/models/WASB-SBDT"
63         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64         conda_env: str = "wasb"
65         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66         sport: str = "badminton"
67         wsl_stage_dir: str = "~/clips"
68         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69         velocity_threshold: float = 0.02
70         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
71         ---
72         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
73         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
74         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
75         device: str = "cuda" # auto | cuda | mps | cpu
76         python_bin: str = "python" # the `wasb` conda env python (invoked by path, no
`conda activate`)
77         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
78         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
79         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
80         keep_frames: bool = False
81         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
82         min_free_gb: float = 0.0
83         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
84         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
85         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
86         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
87         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
88         # Tri-state:
89         # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
90         # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
91         # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
92         stream_video: Optional[bool] = None

```

```

92
93
94 class FusionConfig(BaseModel):
95     """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
96     P2).
97
98     Every default reproduces control behaviour: the fusion segmenter persists the
99     shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
100     is
101     true ? the person-cue features, but it does NOT gate the served handoff unless a
102     threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
103     mask
104     is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
105     only);
106     supplied ? off-court persons (spectators / adjacent courts) are excluded.
107     """
108     # Which trajectory substrate seeds the windows (shuttle stays primary).
109     substrate: Literal["wasb", "tracknet"] = "wasb"
110     # Windowing velocity threshold for the fusion family (the engine reads
111     # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
112     # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
113     velocity_threshold: float = 0.02
114     # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
115     # enabled ? so the default path never imports torch / touches the GPU.
116     cue_flags: dict[str, bool] = Field(default_factory=dict)
117     # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
118     handoff.
119     # 0 = OFF (no gating; persisted candidates are always the full superset for offline
120     re-scoring).
121     min_crossings: int = Field(default=0, ge=0)
122     # Served Gate B: when the person cue is on, drop windows with median in-court
123     players
124     # < this. 0 = OFF.
125     min_players: int = Field(default=0, ge=0)
126     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
127     court_polygon: Optional[list[list[float]]] = None
128     # Net line for the person two-sided-motion split (person features only ? the shuttle
129     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
130     net_axis: Literal["x", "y"] = "y"
131     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
132
133 class IndexingConfig(BaseModel):
134     default_segmenter: str = "yolo_hybrid"
135     use_gpu: bool = True
136     # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
137     # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
138     # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
139     # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
140     # RALLY_DETECTOR_IMPL env var overrides this. See
141     pipeline/detectors/{stub,native_wasb}_runner.py.
142     detector_impl: str = "auto"
143     motion_threshold: float = 0.08
144     min_segment_duration: float = 3.0
145     dead_time_merge_gap: float = 2.0
146     chunk_duration: float = 90.0
147     chunk_overlap: float = 10.0
148     detector_model: str = "fasterrcnn_resnet50_fpn"
149     detector_score_threshold: float = 0.5
150     yolo_velocity_threshold: float = 0.15
151     yolo_frame_skip: int = 3
152     yolo_tracker: str = "bytetrack.yaml"
153     yolo_prefetch_workers: int = 2
154     min_action_window_duration: float = 2.0
155     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0

```

```

= a window
149      # longer than this many seconds is recursively split at its largest internal
inactivity gap (?)
150      # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151      # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152      # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153      # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154      # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155      max_window_duration: float = 6.0
156      window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
point
157      # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158      # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159      # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160      # Only cuts (recall can't drop). OFF by default until measured/promoted.
161      window_hard_cap: bool = False
162      # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163      # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164      # [|?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165      # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166      # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167      # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, [|?end| 4.96?3.31s (n=13).
The W1 cap
168      # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169      window_inactivity_end: float = 1.5
170      inactivity_rally_thresh: float = 0.20
171      inactivity_min_density: float = 0.30
172      # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173      # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174      # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175      # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176      # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177      # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178      # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179      # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180      window_blob_fallback: bool = False
181      # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182      # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183      # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184      # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".

```

```

185     window_blob_factor: float = 4.0
186     log_yolo_candidates: bool = True
187     store_yolo_candidates: bool = True
188     ai_handoff_padding: float = 2.0
189     ai_max_workers: int = 5
190     # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191     # chunks of a high-bitrate (4K) source blow past the provider upload cap
192     # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193     # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194     # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195     # Matches gemini_refine's --clip-scale-width default.
196     ai_chunk_scale_width: int = 1280
197     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202     skip_ai_handoff: bool = False
203     # Optional debug knob: trim every video to the first N seconds before processing.
204     debug_duration: Optional[float] = None
205
206     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208     fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210     @field_validator("default_segmenter")
211     @classmethod
212     def segmenter_must_be_registered(cls, v: str) -> str:
213         # Registry check happens at runtime in main/segmenter selection.
214         # Here we just allow any string for forward compatibility.
215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""

```

```

243     enabled: bool = True
244     text: str = "Generated by Khelsutra.guru"
245     logo_path: str = ""          # optional transparent PNG overlay; "" = no logo
246     font_path: str = ""          # optional .ttf; "" = use the fontconfig `font` family
below
247     font: str = "Sans"           # fontconfig family used when font_path is empty
248     font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249     opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250     box: bool = True             # faint backing box behind the text for legibility
251     margin: int = Field(default=24, ge=0) # px inset from the chosen corner
252     position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-

```

```

Jwt-Assertion`
299     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302     # application's audience tag. NO secrets here ? these are public identifiers.
303     edge_auth_enabled: bool = False
304     cf_team_domain: str = ""          # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305     cf_access_aud: str = ""          # the CF Access application AUD tag the token must
carry
306     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307     # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
308     # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``.
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root

```



```

directly.
341     workspace_prefix: str = "workspaces"
342     local: dict[str, Any] = Field(default_factory=dict)
343     s3: dict[str, Any] = Field(default_factory=dict)
344     gcs: dict[str, Any] = Field(default_factory=dict)
345     gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
$4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
a real one
383         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0 # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):

```

```

388     """Root configuration object.
389
390     All runtime configuration should be accessed via an instance of this class
391     (or the loader) rather than raw dict lookups.
392     """
393
394     sport: Sport = "badminton"
395     ai_provider: str = "gemini"
396     # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
397     # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398     ai_model: str = "gemini-flash-latest"
399
400     validation: ValidationConfig = Field(default_factory=ValidationConfig)
401     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
402     output: OutputConfig = Field(default_factory=OutputConfig)
403     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404     security: SecurityConfig = Field(default_factory=SecurityConfig)
405     logging: LoggingConfig = Field(default_factory=LoggingConfig)
406     storage: StorageConfig = Field(default_factory=StorageConfig)
407     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408     billing: BillingConfig = Field(default_factory=BillingConfig)
409
410     # Allow extra keys during transition so existing config.json files do not break.
411     model_config = {
412         "extra": "allow",
413         "validate_assignment": True,
414     }
415
416     # Temporary migration shim for legacy dict-style access (config.get("section",
{})).get(...)).
417     # Will be removed once all call sites (validators, segmenters, etc.) move to typed
418     # attribute access / the typed config. A nested *section* is returned as a plain
dict
419     # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420     # section model instead breaks it (a model has no .get), which bit every validator
on
421     # real runs. Scalar leaves are returned as-is.
422     def get(self, key: str, default: Any = None) -> Any:
423         _MISSING = object()
424         val: Any = _MISSING
425         if key in type(self).model_fields or hasattr(self, key):
426             val = getattr(self, key, _MISSING)
427         if val is _MISSING:
428             # Legacy convenience: find a leaf key inside any nested section. Sections
are
429             # discovered dynamically from the model fields, so a future top-level
section
430             # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431             for fname in type(self).model_fields:
432                 section = getattr(self, fname, None)
433                 if isinstance(section, BaseModel) and hasattr(section, key):
434                     val = getattr(section, key)
435                     break
436             if val is _MISSING:
437                 return default
438             if isinstance(val, BaseModel):
439                 return val.model_dump(mode="json")
440             return val
441
442     def __getitem__(self, key: str) -> Any:
443         val = self.get(key)
444         if val is None and key not in type(self).model_fields:

```

```

445             # Re-raise like a real dict for unknown top-level
446             raise KeyError(key)
447         return val
448
449
450     # Convenience type alias for callers that want to annotate
451     Config = AppConfig
452

```

10. assistant (2026-06-22T09:08:16.388Z)

Now let me look at pipeline segmenters and the main entry point:

11. user (2026-06-22T09:08:17.883Z)

```

1     import os
2     import sys
3     import argparse
4     import json
5     import logging
6     import shutil
7     import socket
8     import threading
9     import uuid
10    import webbrowser
11    from importlib.metadata import PackageNotFoundError, version as _pkg_version
12    from urllib.parse import quote
13    import uvicorn
14    from dotenv import load_dotenv
15
16    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
17    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
18    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
19    # stdlib-only import ? keeps this pre-pydantic line light.
20    from backend.utils.app_home import env_file_to_load, resolve_output_dir
21
22    load_dotenv(env_file_to_load())
23
24    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
25    logging.basicConfig(
26        level=logging.INFO,
27        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
28        datefmt='%H:%M:%S'
29    )
30    logger = logging.getLogger(__name__)
31    from fastapi import FastAPI, HTTPException, Request
32    from fastapi.middleware.cors import CORSMiddleware
33    from starlette.middleware.trustedhost import TrustedHostMiddleware
34    from pydantic import BaseModel
35    from typing import List, Dict, Any, Optional
36
37    from fastapi.staticfiles import StaticFiles
38    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
39    from backend.pipeline.validators.base import registry
40    from backend.infrastructure.database import Database
41    from backend.pipeline.segmenters.base import segmenter_registry
42    from backend.pipeline.stitcher.compiler import VideoCompiler
43    from backend.pipeline.stitcher.watermark_i18n import localize_watermark
44    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
45    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path

```

```

46     from backend.utils.redact import redact_secrets # P0b: mask secret-shaped fields in
/api/config
47     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
48     from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
49     from backend.config import ( # new typed config
50         load_config as load_app_config,
51         write_light_default_config, # P2b: batteries-included first-run config seeding
52         AppConfig,
53         StitchingConfig,
54         enforce_startup_checks,
55         StartupCheckError,
56     )
57     from backend.results import Failure # standardized error/result contract
58     from backend.reporting import emit_indexer_report
59     from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
60     from backend import billing # M8: per-job cost ledger (USD internal / INR display)
61     from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
62
63     app = FastAPI(title="Sports Highlight Indexer API")
64
65
66     def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
67         """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
68         default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
69         does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
70         src = os.environ if env is None else env
71         raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
72         origins = [o.strip() for o in raw.split(",") if o.strip()]
73         return origins or ["*"]
74
75
76     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
77     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
78     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
79     # lock it to its origin; the middleware is fixed at startup.
80     app.add_middleware(
81         CORSMiddleware,
82         allow_origins=_cors_origins_from_env(),
83         allow_credentials=False,
84         allow_methods=["*"],
85         allow_headers=["*"],
86     )
87
88
89     def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
90         """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
91         web page can point a domain it controls at 127.0.0.1 and script requests to the
local server;
92         the browser sends ``Host: attacker.com``, which this rejects (the server only
answers loopback
93         Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the
default keeps
94         Starlette's TestClient host ``testserver`` so the in-process suite isn't broken.
Read from the
95         ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a

```

```

routable/edge
96         deploy sets its public host (or ``*`` when fronted by an auth gate that validates
the host)."""
97         src = os.environ if env is None else env
98         raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
99         hosts = [h.strip() for h in raw.split(",") if h.strip()]
100        # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
101        # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"),
so IPv6
102        # loopback is NOT supported by this guard ? access the appliance via
127.0.0.1/localhost.
103        return hosts or ["localhost", "127.0.0.1", "testserver"]
104
105
106        # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
honoured
107        # (a remote page can't read responses cross-origin AND its Host header is rejected
here). www_redirect
108        # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
109        app.add_middleware(
110            TrustedHostMiddleware,
111            allowed_hosts=_allowed_hosts_from_env(),
112            www_redirect=False,
113        )
114
115
116        def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] =
None) -> str:
117            """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
118            when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS
an auth gate
119            in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a
routable
120            interface with no auth gate is unsafe, hence the localhost default; widen it
consciously."""
121            if cli_host and cli_host.strip():
122                return cli_host.strip()
123            env = os.environ.get("RALLY_BIND_HOST")
124            if env and env.strip():
125                return env.strip()
126            sec = getattr(config, "security", None)
127            if getattr(sec, "edge_auth_enabled", False):
128                return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
129            return "127.0.0.1"
130
131
132        # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
01) ---
133
134        def _port_is_free(host: str, port: int) -> bool:
135            """True if (host, port) can be bound right now (best-effort; a TOCTOU race is
acceptable ?
136            uvicorn surfaces a real bind failure loudly)."""
137            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
138                try:
139                    s.bind((host, port))
140                    return True
141                except OSError:
142                    return False
143
144
145        def _find_free_port(host: str) -> int:

```

```

146         """An OS-assigned free port on host."""
147         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
148             s.bind((host, 0))
149             return int(s.getsockname()[1])
150

```

12. user (2026-06-22T09:08:19.189Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40             """Registers a new segmenter class under its name property."""
41             try:
42                 temp_inst = segmenter_cls(None, {})
43                 name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49             return self._segmenters.get(name)
50
51         def list_available(self) -> Dict[str, str]:
52             """Returns a dict of registered segmenter names and their descriptions."""
53             res = {}
54             for name, cls in self._segmenters.items():

```

```

55         try:
56             temp_inst = cls(None, {})
57             desc = temp_inst.description
58         except Exception:
59             desc = "No description available."
60         res[name] = desc
61     return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

13. user (2026-06-22T09:08:20.306Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/fusion_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/gemini.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/motion.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/tracknet_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/trajectory_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/wasb_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/yolo_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/_keyhint.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/__init__.py

```

14. assistant (2026-06-22T09:08:22.895Z)

Now let me check the run_regression.py to see the existing regression structure and look at golden fixtures:

15. user (2026-06-22T09:08:23.656Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples
8     -----
9     Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15     import os
16     import sys
17     import json
18     import argparse
19     import logging
20
21     from backend.infrastructure.database import Database
22     from backend.config import load_config

```

```

23     from backend.annotations import annotation_registry
24     from backend.eval import regression
25
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:
33         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35         p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38         p.add_argument("--stage", choices=("candidates", "final"), default="final")
39         p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40         p.add_argument("--iou", type=float, default=0.5)
41         p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42         p.add_argument("--db", default=None)
43         p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44         p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46         p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48         p.add_argument("--json", dest="json_out", default=None)
49         return p
50
51
52     def main(argv=None) -> int:
53         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54         args = build_parser().parse_args(argv)
55
56         db_path = args.db or _default_db_path()
57         if not os.path.exists(db_path):
58             print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59             return 2
60         db = Database(db_path)
61
62         result = regression.regress_with_provider(
63             db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64             stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65             tolerance=args.tolerance, baseline_path=args.baseline,
66         )
67
68         print(regression.format_result(result))
69
70         if args.update_baseline:
71             regression.save_baseline(result.video_id, result.stage, result.precision,
72                                     result.recall, result.f1, path=args.baseline,
73                                     iou_threshold=result.iou_threshold,
74                                     gt_fingerprint=result.gt_hash)
75             print(f"Baseline updated in {args.baseline}.")
76
77         if args.json_out:

```



```

78         with open(args.json_out, "w") as f:
79             json.dump(result.as_dict(), f, indent=2)
80             print(f"JSON written to {args.json_out}")
81
82         # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83         if result.regressed:
84             return 1
85         if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86             print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87
88                 "Snapshot one with --update-baseline (then commit it), or pass "
89                 "--allow-missing-baseline to accept this run.", file=sys.stderr)
90             return 3
91         return 0
92
93 if __name__ == "__main__":
94     raise SystemExit(main())
95

```

16. user (2026-06-22T09:08:24.688Z)

```

1      [
2      {
3          "fps": 30.0,
4          "frame_width": 1280.0,
5          "kind": "synthetic",
6          "paths": {
7              "dir": "clean_single_rally",
8              "expected": "clean_single_rally/expected.json",
9              "labels": "clean_single_rally/labels.csv",
10             "provenance": "clean_single_rally/provenance.json",
11             "trajectory": "clean_single_rally/trajectory.csv"
12         },
13         "schema_version": 1,
14         "slug": "clean_single_rally",
15         "windowing": {
16             "max_win": 0.0,
17             "mg": 2.0,
18             "mwd": 2.0,
19             "vt": 0.02
20         }
21     },
22     {
23         "fps": 30.0,
24         "frame_width": 1280.0,
25         "kind": "synthetic",
26         "paths": {
27             "dir": "multi_rally_with_deadtime",
28             "expected": "multi_rally_with_deadtime/expected.json",
29             "labels": "multi_rally_with_deadtime/labels.csv",
30             "provenance": "multi_rally_with_deadtime/provenance.json",
31             "trajectory": "multi_rally_with_deadtime/trajectory.csv"
32         },
33         "schema_version": 1,
34         "slug": "multi_rally_with_deadtime",
35         "windowing": {
36             "max_win": 0.0,
37             "mg": 2.0,
38             "mwd": 2.0,
39             "vt": 0.02
40         }
41     },
42     {

```

```

43     "fps": 30.0,
44     "frame_width": 1280.0,
45     "kind": "synthetic",
46     "paths": {
47         "dir": "occlusion_break",
48         "expected": "occlusion_break/expected.json",
49         "labels": "occlusion_break/labels.csv",
50         "provenance": "occlusion_break/provenance.json",
51         "trajectory": "occlusion_break/trajectory.csv"
52     },
53     "schema_version": 1,
54     "slug": "occlusion_break",
55     "windowing": {
56         "max_win": 0.0,
57         "mg": 2.0,
58         "mwd": 2.0,
59         "vt": 0.02
60     }
61 }
62 ]
63

```

17. user (2026-06-22T09:08:25.688Z)

```

total 48
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 22 14:36 experiment_leaderboard.json
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 fixtures
-rw-r--r-- 1 avidu 197609 1977 Jun 22 14:36 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 22 14:36 heuristic_lovo_n6.json
-rw-r--r-- 1 avidu 197609 17458 Jun 22 14:36 nightly_baseline.json

```

18. assistant (2026-06-22T09:08:27.736Z)

Now let me check the nightly baseline and understand the structure better:

19. user (2026-06-22T09:08:29.189Z)

```

1  {
2      "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3      "configs": {
4          "baseline_off": {
5              "aggregate": {
6                  "dend_abs_median": 41.77921047714383,
7                  "dstart_abs_median": 42.467243955066174,
8                  "f1@0.25": 0.39981006941991765,
9                  "f1@0.5": 0.2366145803119009,
10                 "merge_rate": 0.6380248744753217,
11                 "precision": 0.2577557018251538,
12                 "recall": 0.22719343607869802,
13                 "segment_count_ratio": 0.5824947574930804,
14                 "split_rate": 0.0,
15                 "tol_f1": 0.1382064040388172,
16                 "tol_precision": 0.14536686157033415,
17                 "tol_recall": 0.13445461315285676
18             },
19             "iou": 0.5,
20             "min_crossings": 0,
21             "n": 15,
22             "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",

```

```

23     "per_video": {
24         "Badminton_BXH_2": {
25             "f1@0.25": 0.0,
26             "f1@0.5": 0.0,
27             "merge_count": 3,
28             "merge_rate": 1.0,
29             "num_gt": 44,
30             "num_pred": 4,
31             "split_count": 0,
32             "tol_f1": 0.0
33         },
34         "Boxhill_Doubles": {
35             "f1@0.25": 0.10526315789473685,
36             "f1@0.5": 0.03508771929824561,
37             "merge_count": 7,
38             "merge_rate": 0.9534883720930233,
39             "num_gt": 43,
40             "num_pred": 14,
41             "split_count": 0,
42             "tol_f1": 0.0
43         },
44         "GX010128": {
45             "f1@0.25": 0.14925373134328357,
46             "f1@0.5": 0.05970149253731343,
47             "merge_count": 12,
48             "merge_rate": 0.9615384615384616,
49             "num_gt": 52,
50             "num_pred": 15,

```

20. user (2026-06-22T09:08:30.422Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts

21. user (2026-06-22T09:08:31.568Z)

```

total 108
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609 1391 Jun 22 14:36 README.md
-rwxr-xr-x 1 avidu 197609 4556 Jun 22 14:36 bundle_ui.sh
-rw-r--r-- 1 avidu 197609 5796 Jun 22 14:36 doctor.py
-rwxr-xr-x 1 avidu 197609 10154 Jun 22 14:36 install.py
-rw-r--r-- 1 avidu 197609 3606 Jun 22 14:36 nightly_regression.ps1
-rw-r--r-- 1 avidu 197609 30779 Jun 22 14:36 nightly_regression.py
-rw-r--r-- 1 avidu 197609 3466 Jun 22 14:36 register_nightly_task.ps1
-rw-r--r-- 1 avidu 197609 8514 Jun 22 14:36 run_phase3_eval.py
-rw-r--r-- 1 avidu 197609 7616 Jun 22 14:36 run_process_and_stitch.py
-rwxr-xr-x 1 avidu 197609 7594 Jun 22 14:36 uninstall.py

```

22. assistant (2026-06-22T09:08:33.883Z)

Perfect! Now let me examine the nightly_regression.py script:

23. user (2026-06-22T09:08:34.986Z)

```

1     """Nightly rally-quality regression tripwire.
2
3     Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4     seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
5     diffs
6     the headline metrics against a FROZEN baseline
7     (`eval_baselines/nightly_baseline.json`).
8     Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
9     video

```

```

7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42     python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,

```

```

52     but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os
60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
80     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92     )
93     #: Per-video metrics snapshotted into the baseline.
94     PV_SNAPSHOT: Tuple[str, ...] = (
95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96     )
97
98     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float
jitter)
99     DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
101
102
103     # ----- #
104     # The two tracked configs.
105     # ----- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110         name: str

```

```

111     preset: Any          # backend.eval.windowing.WindowingPreset
112     min_crossings: int = 0
113     note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119         All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120         min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122         * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123         point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124         DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125         knob.
126         * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127         the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128         what we expect?" stays answerable even if a future change moves DEFAULT.
129         * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130         the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131         dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132         launch-time value). It is the reference DEFAULT used to be before the flip."""
133         from backend.eval.windowing import SERVED
134
135         default = NightlyConfig(
136             name="default", preset=SERVED, min_crossings=0,
137             note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138         )
139         milestone = NightlyConfig(
140             name="milestone",
141             preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                           inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                           inactivity_min_density=0.30),
144             min_crossings=0,
145             note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
146         )
147         baseline_off = NightlyConfig(
148             name="baseline_off",
149             preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
150                           inactivity_end=0.0, inactivity_rally_thresh=0.08,
151                           inactivity_min_density=0.34,
152                           blob_fallback=False, blob_factor=4.0),
153             min_crossings=0,
154             note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
155         )
156         return [default, milestone, baseline_off]
157
158     # ----- #
159     # Tolerances bundle.
160     # ----- #

```

```

161 @dataclass(frozen=True)
162 class Tolerances:
163     metric_tol: float = DEFAULT_METRIC_TOL
164     rate_tol: float = DEFAULT_RATE_TOL
165     over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167     def to_dict(self) -> Dict[str, Any]:
168         return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
169             "over_seg_tol": self.over_seg_tol}
170
171
172 # ----- #
173 # Pure summary + comparison core (no I/O ? unit-testable).
174 # ----- #
175 def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176     """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178     Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179     which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180     agg = eval_result.get("aggregate", {}) or {}
181     aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182     per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183     return {
184         "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185         "preset": eval_result.get("preset"),
186         "min_crossings": eval_result.get("min_crossings", 0),
187         "iou": eval_result.get("iou", DEFAULT_IOU),
188         "aggregate": aggregate,
189         "per_video": per_video,
190     }
191
192
193 @dataclass
194 class Finding:
195     """One comparison outcome line (regression / improvement / stale note)."""
196
197     config: str
198     scope: str          # "aggregate" | "per_video" | "config"
199     metric: str
200     kind: str           # "regression" | "improvement" | "stale"
201     baseline: Optional[float] = None
202     current: Optional[float] = None
203     video: Optional[str] = None
204
205     @property
206     def delta(self) -> Optional[float]:
207         if self.baseline is None or self.current is None:
208             return None
209         return self.current - self.baseline
210
211     def message(self) -> str:
212         loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213         if self.kind == "stale":
214             return f"[STALE] {self.config}: {self.metric}"
215         d = self.delta
216         ds = f"{d:+.4f}" if d is not None else "?"
217         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218         c = "?" if self.current is None else f"{self.current:.4f}"
219         tag = "REGRESSION" if self.kind == "regression" else "improvement"
220         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221

```

```

222
223 @dataclass
224 class ConfigComparison:
225     config: str
226     status: str = "ok" # "ok" | "regression" | "stale"
227     findings: List[Finding] = field(default_factory=list)
228     added_videos: List[str] = field(default_factory=list)
229     removed_videos: List[str] = field(default_factory=list)
230     aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
comparable
231
232 @property
233 def regressions(self) -> List[Finding]:
234     return [f for f in self.findings if f.kind == "regression"]
235
236 @property
237 def improvements(self) -> List[Finding]:
238     return [f for f in self.findings if f.kind == "improvement"]
239
240
241 def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242     """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243     if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244         return base_preset == cur_preset
245     keys = set(base_preset) | set(cur_preset)
246     return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248
249 def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                  tols: Tolerances) -> ConfigComparison:
251     """Compare one config's current summary to its baseline summary.
252
253     * If the preset definition or the net-crossings gate changed, the whole config is
STALE
254     (numbers aren't comparable) ? flag it, skip gating.
255     * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
256     skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
257     INTERSECTION (a code regression on a shared video must still trip the wire).
258     """
259     cmp = ConfigComparison(config=name)
260
261     # 1) Config-definition drift ? stale, not gateable.
262     if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
263         or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
264         cmp.status = "stale"
265         cmp.aggregate_gated = False
266         cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
267     return cmp
268
269     base_pv = base_sum.get("per_video", {})
270     cur_pv = cur_sum.get("per_video", {})
271     base_names, cur_names = set(base_pv), set(cur_pv)
272     cmp.added_videos = sorted(cur_names - base_names)
273     cmp.removed_videos = sorted(base_names - cur_names)
274     corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
275
276     # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277     if corpus_changed:
278         cmp.aggregate_gated = False
279         cmp.status = "stale"

```



```

280         cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281     else:
282         base_agg = base_sum.get("aggregate", {})
283         cur_agg = cur_sum.get("aggregate", {})
284         for m in GATED_HIGHER:
285             b, c = base_agg.get(m), cur_agg.get(m)
286             if b is None or c is None:
287                 continue
288             if c < b - tols.metric_tol:
289                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290             elif c > b + tols.metric_tol:
291                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292         for m in GATED_LOWER:
293             b, c = base_agg.get(m), cur_agg.get(m)
294             if b is None or c is None:
295                 continue
296             if c > b + tols.rate_tol:
297                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298             elif c < b - tols.rate_tol:
299                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
302     for vid in sorted(base_names & cur_names):
303         for m in PER_VIDEO_GUARD:
304             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
305             if b is None or c is None:
306                 continue
307             if c > b + tols.over_seg_tol:
308                 cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310     if cmp.regressions:
311         cmp.status = "regression"
312     elif cmp.status != "stale":
313         cmp.status = "ok"
314     return cmp
315
316
317 @dataclass
318 class NightlyResult:
319     comparisons: List[ConfigComparison] = field(default_factory=list)
320     skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
videos not scored
321
322     @property
323     def has_regression(self) -> bool:
324         return any(c.status == "regression" for c in self.comparisons)
325
326     @property
327     def has_stale(self) -> bool:
328         return any(c.status == "stale" for c in self.comparisons)
329
330     def overall_status(self) -> str:
331         if self.has_regression:
332             return "REGRESSION"
333         if self.has_stale:
334             return "STALE"
335         return "PASS"
336
337     def exit_code(self) -> int:
338         if self.has_regression:
339             return 1
340         if self.has_stale:

```

```

341         return 4
342     return 0
343
344
345     def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
346                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
347         """Compare every tracked config's current summary to the baseline summary."""
348         res = NightlyResult(skipped=skipped or {})
349         base_cfgs = baseline.get("configs", {})
350         cur_cfgs = current.get("configs", {})
351         for name in cur_cfgs:
352             if name not in base_cfgs:
353                 cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354                 cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355                 res.comparisons.append(cmp)
356                 continue
357             res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358         return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366
367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                      result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"**Status: {badge}**   · exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"HEAD: `{current.get('git_commit', '?')}` · baseline: "
378                f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}` ·
IoU={current.get('iou', DEFAULT_IOU)}")
380         L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
381                f"per-video over-seg ±{tols.over_seg_tol}")
382         L.append("")
383
384         # Up-front regression / stale summary.
385         regs = [f for c in result.comparisons for f in c.regressions]
386         if regs:
387             L.append("## ? Regressions")
388             L.append("")
389             for f in regs:
390                 L.append(f"- {f.message()}")
391             L.append("")
392         if result.has_stale:
393             L.append("## ? Stale / not comparable")
394             L.append("")
395             for c in result.comparisons:
396                 if c.status == "stale":
397                     if c.added_videos:
398                         L.append(f"- `{c.config}`: corpus added {c.added_videos}")
399                     if c.removed_videos:

```

```

400         L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
401         if not c.added_videos and not c.removed_videos:
402             L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403             L.append("- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404             L.append("")
405
406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("### ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- `{cfg}`: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"### Config `{cmp.config}` ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})")
424             L.append("")
425             cagg = cur.get("aggregate", {})
426             bagg = base.get("aggregate", {})
427             L.append("| metric | baseline | current | ? | gated |")
428             L.append("|---|---|---|---|---|")
429             for m in AGG_SNAPSHOT:
430                 b, c = bagg.get(m), cagg.get(m)
431                 d = (c - b) if (b is not None and c is not None) else None
432                 gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433                 L.append(f"| {m} | {fmt(b)} | {fmt(c)} | {fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("|---|---|---|---|")
442                 for v in names:
443                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
444                     tf = f"{fmt(cb.get('tol_f1'))}?{fmt(cc.get('tol_f1'))}"
445                     sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
446                     mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"
447                     flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
448                     L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
449                 L.append("")
450             imps = cmp.improvements
451             if imps:
452                 L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453                 for f in imps:
454                     L.append(f"- {f.message()}")
455                 L.append("")
456
457         L.append("----")

```

```

458         L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
459             "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460         return "\n".join(L)
461
462
463     # ----- #
464     # I/O shell.
465     # ----- #
466     def _git_commit() -> str:
467         import subprocess
468         try:
469             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
470                                 capture_output=True, text=True, timeout=10)
471             return out.stdout.strip() or "?"
472         except Exception:
473             return "?"
474
475
476     def _display_path(path: str) -> str:
477         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
478         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
479         Metadata only ? the comparison never reads it."""
480         norm = path.replace("\\", "/")
481         parent = os.path.basename(os.path.dirname(norm))
482         if parent == "output":
483             return "output/" + os.path.basename(norm)
484         try:
485             rel = os.path.relpath(path, REPO_ROOT)
486             if not rel.startswith(".."):
487                 return rel.replace("\\", "/")
488         except ValueError:
489             pass
490         return norm
491
492
493     def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
494                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
495         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
496
497         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
498         ``skipped``
499         lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
500         surfaced in
501         the report so the corpus can't silently shrink."""
502         from backend.eval import rally_seg_eval
503
504         golden = rally_seg_eval.load_golden(manifest, features_dir)
505         manifest_names = [v["name"] for v in golden]
506         if not golden:
507             raise RuntimeError(f"no golden videos loaded from {manifest}")
508
509         snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
510         skipped: Dict[str, List[str]] = {}
511         scored_any = False
512         for cfg in configs:
513             result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
514             summary = summarize_run(result)
515             summary["note"] = cfg.note

```

```

514         snapshot["configs"][cfg.name] = summary
515         scored = set(summary["per_video"])
516         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
517         scored_any = scored_any or bool(scored)
518     if not scored_any:
519         raise RuntimeError(
520             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
521             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"")
522     return snapshot, skipped
523
524
525     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
526         if not os.path.exists(path):
527             return None
528         with open(path, encoding="utf-8") as fh:
529             return json.load(fh)
530
531
532     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
533         out = dict(snapshot)
534         out["generated_at"] = _dt.date.today().isoformat()
535         out["git_commit"] = _git_commit()
536         out["tolerances"] = tols.to_dict()
537         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
538             "`python scripts/nightly_regression.py --update-baseline` after an
intended "
539             "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
540         os.makedirs(os.path.dirname(path), exist_ok=True)
541         tmp = path + ".tmp"
542         with open(tmp, "w", encoding="utf-8") as fh:
543             json.dump(out, fh, indent=2, sort_keys=True)
544             fh.write("\n")
545         os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
(offline, CPU).")
550         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
551         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
552             help="dir holding <name>_golden_features.json")
553         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
path")
554         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
are written")
555         p.add_argument("--iou", type=float, default=DEFAULT_IOU)
556         p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
557             help="F1-like aggregate drop that counts as a regression")
558         p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
559             help="merge_rate rise that counts as a regression")
560         p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
561             help="per-video split/merge-count increase allowed before it's a
regression")
562         p.add_argument("--update-baseline", action="store_true",
563             help="snapshot the current numbers as the new baseline instead of
checking")
564         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
565         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
566         return p
567
568

```

```

569     def main(argv: Optional[List[str]] = None) -> int:
570         import logging
571         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
572         log = logging.getLogger("nightly_regression")
573         args = build_parser().parse_args(argv)
574         tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
over_seg_tol=args.over_seg_tol)
575         date = args.date or _dt.date.today().isoformat()
576
577         if not os.path.exists(args.manifest):
578             log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
579             return 2
580
581         try:
582             snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
583         except RuntimeError as e:
584             log.error("%s", e)
585             return 2
586         snapshot["git_commit"] = _git_commit()
587         snapshot["generated_at"] = date
588
589         if args.update_baseline:
590             save_baseline(args.baseline, snapshot, tols)
591             log.info("baseline snapshot written to %s", args.baseline)
592             for cfg, s in snapshot["configs"].items():
593                 agg = s["aggregate"]
594                 log.info(" %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
595                     cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
596                     agg.get("merge_rate") or 0.0)
597             return 0
598
599         baseline = load_baseline(args.baseline)
600         if baseline is None:
601             log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
602             return 3
603
604         result = compare_all(baseline, snapshot, tols, skipped)
605         report = format_report(baseline, snapshot, result, tols, date)
606
607         if not args.no_report:
608             os.makedirs(args.report_dir, exist_ok=True)
609             md_path = os.path.join(args.report_dir, f"{date}.md")
610             json_path = os.path.join(args.report_dir, f"{date}.json")
611             with open(md_path, "w", encoding="utf-8") as fh:
612                 fh.write(report + "\n")
613             with open(json_path, "w", encoding="utf-8") as fh:
614                 json.dump({"date": date, "status": result.overall_status(),
615                     "exit_code": result.exit_code(), "current": snapshot,
616                     "regressions": [f.message() for c in result.comparisons for f in
c.regressions]}},
617                 fh, indent=2, sort_keys=True)
618             log.info("report written to %s", md_path)
619
620         print(report)
621         code = result.exit_code()
622         if code == 1:
623             log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                 sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625         elif code == 4:
626             log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627         else:

```

```

628         log.info("nightly regression check PASSED")
629     return code
630
631
632     if __name__ == "__main__":
633         raise SystemExit(main())
634

```

24. assistant (2026-06-22T09:08:37.786Z)

Excellent. Now let me examine how the pipeline is invoked and look at the default config more closely:

25. user (2026-06-22T09:08:40.014Z)

```

1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11      This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12      module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13      (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14      profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15      today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16      spend is never entered silently).
17      """
18
19      from __future__ import annotations
20
21      import copy
22      import os
23      from typing import Any, Mapping, Optional
24
25      # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26      # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27      PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28      COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30      # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31      # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32      COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33          "truly-local": "local_gpu",
34          "cloud-serving": "cloud_run",
35          "cpu-mock": "cpu_mock",
36      }
37
38      # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39      # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40      # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41      # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,

```

```

42 # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43 PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44     "truly-local": {
45         "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46         # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47         # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48         "indexing": {"skip_ai_handoff": True},
49         "storage": {"backend": "local"},
50     },
51     "cloud-serving": {
52         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99

```



```

100     Exposed for the per-profile startup checks (`backend/config/startup.py`) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""
106     return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:
112         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114         Signals, in priority order:
115
116         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117         - ``CI`` truthy                           ? ``"cpu-mock"``
118         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119         - otherwise                               ? ``""`` (no strong suggestion)
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
122         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
123         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
124         """
125         e = os.environ if env is None else env
126         if e.get("K_SERVICE"):
127             return "cloud-serving"
128         if _truthy(e.get("CI")):
129             return "cpu-mock"
130         if probe_cuda and _cuda_available():
131             return "truly-local"
132         return ""
133

```

26. user (2026-06-22T09:08:41.631Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/loader.py:# * default_segmenter "motion"
? pure-OpenCV, no torch / GPU / weights / API key (LIGHT tier);
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/loader.py: "indexing":
{"default_segmenter": "motion", "use_gpu": False},
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/models.py: default_segmenter: str =
"yolo_hybrid"
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/config/models.py:
@field_validator("default_segmenter")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/main.py: "default_segmenter":
getattr(indexing, "default_segmenter", None),
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/main.py: "default_segmenter":
getattr(indexing, "default_segmenter", None),
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/main.py: default_seg =

```

```

getattr(getattr(global_config, "indexing", None), "default_segmenter", "motion")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/main.py:          seg_name = args.segmenter or
getattr(getattr(global_config, "indexing", None), "default_segmenter", "motion")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/reporting/harvest.py:    config_default =
(config.get("indexing") or {}).get("default_segmenter")
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/worker/__main__.py:          seg_name =
args.segmenter or config.indexing.default_segmenter
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/worker/__main__.py:    pi.add_argument("--
segmenter", default=None, help="default: indexing.default_segmenter")

```

27. assistant (2026-06-22T09:08:43.751Z)

Now let me check the WASB runner to understand checkpoint/weights loading:

28. user (2026-06-22T09:08:45.116Z)

```

1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug

```

```

only)
42     min_free_gb: float = 0.0    # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WSLCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "
76                   f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:
99                 logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):

```

```

102         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103         return None
104         return dest
105
106     def _stage_infer_script(self) -> bool:
107         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108         src_mnt = to_wsl_mnt_path(_INFER_WIN)
109         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110         res = self._wsl_bash(cmd)
111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                    video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119         can never reuse each other's staged frames, resumable cache, or CSV.
120
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149             if free is not None and free < self.cfg.min_free_gb:
150                 logger.warning(

```

29. user (2026-06-22T09:08:46.619Z)

```

49:         "weights_path": "",
61:         "weights_path": "~/.models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",

```

30. user (2026-06-22T09:08:47.570Z)

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/utils/app_home.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/utils/ffmpeg.py

```

```
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/geometry.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/hashing.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/logging_setup.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/paths.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/redact.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/run_telemetry.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/single_instance.py  
C:/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/backend/utils/video.py
```

31. assistant (2026-06-22T09:08:50.144Z)

Let me check app_home.py and look at how the pipeline is actually started with videos:

32. user (2026-06-22T09:08:50.891Z)

```
1      """Resolved application-data HOME for a packaged, batteries-included install.  
2  
3      PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``,  
4      the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and  
5      ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it  
6      from  
7      the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user  
8      silently  
9      "loses" their index. This module introduces ONE resolver every state path roots on.  
10     Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:  
11     * ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher  
12     sets it).  
13     * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a  
14     behaviour.  
15     The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` /  
16     ``~/local/share``)  
17     is computed by :func:`default_os_app_home` but is **NOT** auto-selected here ? the  
18     launcher (P2)  
19     calls it and exports ``RALLY_APP_HOME`` explicitly. Keeping the resolver opt-in means  
20     importing  
21     this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise  
22     writes into a  
23     real user profile during a test run).  
24     stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ?  
25     before  
26     the heavier pydantic-backed ``backend.config`` package ? keeping the early  
27     ``load_dotenv`` path light.  
28     """  
29     from __future__ import annotations  
30  
31     import os  
32     import platform  
33     from pathlib import Path  
34  
35     #: Environment variable the packaged launcher sets to pin the app-data home.  
36     APP_HOME_ENV = "RALLY_APP_HOME"  
37  
38     #: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
```

```

33     APP_DIR_NAME = "Khelsutra"
34
35
36     def app_home() -> Path:
37         """The resolved application-data home.
38
39         ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD,
byte-identical
40         to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
41         """
42         env = os.environ.get(APP_HOME_ENV)
43         if env and env.strip():
44             return Path(env.strip()).expanduser()
45         return Path(".")
46
47
48     def app_path(*parts: str | os.PathLike[str]) -> Path:
49         """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
50         return app_home().joinpath(*[str(p) for p in parts])
51
52
53     def default_config_path() -> Path:
54         """``<app-home>/config.json`` ? the config file the loader reads and ``--setup``
writes.
55
56         With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative),
unchanged.
57         """
58         return app_home() / "config.json"
59
60
61     def default_env_path() -> Path:
62         """``<app-home>/env`` ? the dotenv file the server prefers (e.g.
``GEMINI_API_KEY``)."""
63         return app_home() / ".env"
64
65
66     def env_file_to_load() -> Path | None:
67         """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's
DEFAULT
68         (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()``
behaviour.
69
70         Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file
exists. With the
71         env UNSET this returns ``None`` so the caller keeps the historical search and does
NOT newly
72         prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file,
not the CWD).
73         """
74         home = app_home()
75         if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv()
search
76             return None
77         candidate = home / ".env"
78         return candidate if candidate.exists() else None
79
80
81     def resolve_output_dir(output_dir: str) -> str:
82         """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one
through.
83
84         With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default
``"output"`` when
85         empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no

```

```

pathlib
86         normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir
like
87         ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the
rest of the
88         app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` /
``C:x``) ? passes
89         through untouched.
90         ""
91         home = app_home()
92         if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
93             return output_dir or "output"
94         if not output_dir:
95             return str(home / "output")
96         if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
97             return output_dir
98         return str(home / output_dir)
99
100
101     def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
102         """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
103         ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).
104
105         * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
106         * macOS   ? ``~/Library/Application Support/<app_name>``
107         * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if
unset)
108         ""
109         system = platform.system()
110         if system == "Windows":
111             base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
112         elif system == "Darwin":
113             base = str(Path.home() / "Library" / "Application Support")
114         else:
115             base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
116         return Path(base) / app_name
117

```

33. user (2026-06-22T09:08:51.838Z)

```

150
151
152     def _resolve_app_port(host: str, port: int) -> int:
153         """For --app: keep `port` if free, else fall back to a free one so a busy 8000 never
blocks the app."""
154         return port if _port_is_free(host, port) else _find_free_port(host)
155
156
157     def _open_browser_when_ready(url: str, health_url: str, timeout: float = 60.0) -> bool:
158         """Poll `health_url` until the server is up, then open `url` in the user's default
browser.
159         Returns True if a browser open was attempted. No-op (returns False) when
RALLY_NO_BROWSER is set
160         (tests / headless) or the server never becomes healthy (don't open a dead page).
Never raises."""
161         if os.environ.get("RALLY_NO_BROWSER"):
162             return False
163         import time as _time
164         import urllib.request
165         deadline = _time.monotonic() + timeout
166         while _time.monotonic() < deadline:
167             try:
168                 with urllib.request.urlopen(health_url, timeout=2) as r:
169                     if getattr(r, "status", 200) == 200:
170                         break

```

```

171         except Exception:
172             _time.sleep(0.3)
173     else:
174         return False
175     try:
176         webbrowser.open(url)
177     except Exception:
178         pass
179     return True
180
181
182     # Serves static frontend files directly (now under api/ per approved structure).
183     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
184     # an arbitrary directory finds the shipped assets instead of creating a stray
185     # ./backend/api/static.
186     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
187     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
188     try:
189         os.makedirs(_STATIC_DIR, exist_ok=True)
190     except OSError:
191         # The dir ships with the package (a no-op create); guard against a read-only install
192         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
193         # PermissionError.
194         pass
195     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
196
197     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
198     # snapshot
199     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
200     # if it's
201     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
202     # no SPA change
203     # is needed; it is query-param driven (?video=?), so serving index.html at `/` + /assets
204     # is enough
205     # (no client-side path routing ? no history-fallback catch-all, which would also be a
206     # route-order hazard).
207     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
208     _UI_DIR = _BUNDLED_UI_DIR if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
209     else _STATIC_DIR
210     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
211     if os.path.isdir(_UI_ASSETS_DIR):
212         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
213
214
215
216     @app.get("/", response_class=HTMLResponse)
217     def serve_index():
218         index_path = os.path.join(_UI_DIR, "index.html")
219         if os.path.exists(index_path):
220             with open(index_path, "r", encoding="utf-8") as f:
221                 return f.read()
222         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
223
224
225
226     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
227         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
228         bundled.
229         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
230         guard)."""
231         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
232         try:
233             with open(path, "r", encoding="utf-8") as f:
234                 return json.load(f)
235         except (OSError, ValueError):
236             return None

```



```

225
226     # Global variables initialized at startup
227     db_instance: Optional[Database] = None
228     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
229     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
230     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
231     _instance_lock: Optional[Any] = None
232
233     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
234     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
235     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
236     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
237     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
238     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
239     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
240     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
241         JobWaiting,
242         _arm_wake_timer,
243         _drain_due_jobs,
244         _get_executor,
245         _job_to_request,
246         _MAX_DRAIN_WAKE_SEC,
247         _maybe_record_job_cost,
248         _run_claimed_job,
249         _schedule_next_drain_if_waiting,
250     )
251
252     # Legacy thin wrapper for any remaining direct calls during transition.
253     # New code should import load_app_config / AppConfig from backend.config directly.
254     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
255         cfg = load_app_config(config_path)
256         return cfg.model_dump(mode="json")
257
258     # --- API Models ---
259     class ProcessRequest(BaseModel):
260         video_path: str
261         player_pool: Optional[List[str]] = None
262         max_frames: Optional[int] = None
263         segmenter: Optional[str] = None
264         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
265         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
266         locale: Optional[str] = None
267
268     class QueryRequest(BaseModel):
269         video_id: str
270         server: Optional[str] = None
271         receiver: Optional[str] = None
272         duration_min: Optional[float] = None
273         event_type: Optional[str] = None
274
275     class CompileRequest(BaseModel):
276         video_id: str
277         segment_ids: List[int]
278         output_filename: str
279         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
280         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).

```

```

281         locale: Optional[str] = None
282
283     def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
284         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
285
286         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
287         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
288         and
289         keep the prior good results intact ? a failed/empty re-run never destroys prior
290         data.
291         """
292         if segments:
293             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294         else:
295             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
296
297     # --- API Endpoints ---
298     @app.get("/api/config")
299     def get_config():
300         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
301         # field. Today
302         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
303         # id), so this
304         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
305         # added (guarded
306         # by tests/test_redact.py + the /api/config redaction test).
307         if global_config is None:
308             return {}
309         return redact_secrets(global_config.model_dump(mode="json"))
310
311     @app.get("/api/health")
312     def health():
313         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
314         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
315         confirm the engine is reachable and which detector is wired. Never raises."""
316         sport = getattr(global_config, "sport", None)
317         indexing = getattr(global_config, "indexing", None)
318         return {
319             "status": "ok",
320             "sport": sport,
321             "default_segmenter": getattr(indexing, "default_segmenter", None),
322         }
323
324     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
325     # existing
326     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
327     # no bump).
328     API_VERSION = 1
329
330     def _engine_version() -> str:
331         """Installed package version (wheel/editable); 'unknown' if not importable as a
332         distribution."""
333         try:
334             return _pkg_version("badminton-highlight-indexer")
335         except PackageNotFoundError:
336             return "unknown"
337
338     @app.get("/api/version")
339     def api_version():
340         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
341         P1)."""

```

```

337         return {"engine_version": _engine_version(), "api_version": API_VERSION, "ui":
bundled_ui_version()}
338
339
340     @app.get("/api/capabilities")
341     def capabilities():
342         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
343         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
344         value. Best-effort + never raises (the wizard degrades gracefully)."""
345         indexing = getattr(global_config, "indexing", None)
346         deployment = getattr(global_config, "deployment", None)
347         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
348         ffprobe = shutil.which(ffprobe_bin())
349         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
350         wasb = getattr(indexing, "wasb", None)
351         weights_path = os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "")
or ""
352         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
353         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
354         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
355         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
356         nvidia_smi = shutil.which("nvidia-smi")
357         return {
358             "api_version": API_VERSION,
359             "engine_version": _engine_version(),
360             "segmenters": sorted(segmenter_registry.list_available().keys()),
361             "default_segmenter": getattr(indexing, "default_segmenter", None),
362             "ffmpeg": {"ffmpeg": bool(ffmpeg), "ffprobe": bool(ffprobe),
363                 "ffmpeg_path": ffmpeg, "ffprobe_path": ffprobe},
364             "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
365             "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
366             "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
367             "deployment": {
368                 "profile": getattr(deployment, "profile", "") or "",
369                 "compute_target": getattr(deployment, "compute_target", "") or "",
370             },
371         }
372
373
374     @app.get("/api/videos")
375     def list_videos(limit: Optional[int] = None):
376         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
377         single-row get existed (PACKAGING_PLAN.md P1)."""
378         return {"videos": db_instance.list_videos(limit=limit)}
379
380
381     def _reels_dir() -> str:
382         return getattr(getattr(global_config, "output", None), "output_dir", "output") or
"output"
383
384
385     _REEL_EXTS = (".mp4", ".mov")
386
387
388     @app.get("/api/reels")
389     def list_reels():
390         """List compiled highlight reels present in the output dir, newest first, with a
download URL.

```

```

391
392     NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
393     does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
394     this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
395     later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
396     base = _reels_dir()
397     reels = []
398     try:
399         entries = os.scandir(base)
400     except OSError:
401         return {"reels": []}
402     with entries:
403         for e in entries:
404             if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
405                 try:
406                     st = e.stat()
407                 except OSError:
408                     continue
409                 reels.append({
410                     "filename": e.name,
411                     "size_bytes": st.st_size,
412                     "modified_at": st.st_mtime,
413                     "download_url": f"/api/reels/{quote(e.name)}",
414                 })
415     reels.sort(key=lambda r: r["modified_at"], reverse=True)
416     return {"reels": reels}
417
418
419     @app.get("/api/reels/{filename}")
420     def download_reel(filename: str):
421         """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
422         Mirrors /api/highlights' safety (bare-name + containment checks)."""
423         try:
424             name = safe_output_filename(filename)
425         except ValueError as e:
426             raise HTTPException(status_code=400, detail=str(e)) from e
427         base = _reels_dir()
428         path = os.path.join(base, name)
429         try:
430             path = resolve_under_root(path, base)
431         except ValueError as e:
432             raise HTTPException(status_code=400, detail=str(e)) from e
433         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
434             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
435         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
436         return FileResponse(path, media_type=media, filename=name)
437
438
439     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
the sync endpoint used to return (UI compatibility); the per-video observability
report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
(after this function has already restored the pre-run segments, see below).
449

```

34. assistant (2026-06-22T09:08:53.936Z)

Now let me look at the CLI entry point to understand the headless recipe:

35. user (2026-06-22T09:08:55.534Z)

```
440         """The full hash ? validate ? segment ? report pipeline for one video.
441
442         This is the former synchronous /api/process body, unchanged in behavior, now run on
443         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
444         the sync endpoint used to return (UI compatibility); the per-video observability
445         report is still emitted in `finally:` and grafted as response["reports"].
446
447         `progress` is an optional callable(stage: str) used to surface coarse job progress.
448         Raises on unexpected internal errors ? the caller records those as a job failure
449         (after this function has already restored the pre-run segments, see below).
450         """
451         notify = progress or (lambda stage: None)
452
453         notify("hashing")
454         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
455         (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
456         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
457         # consumers (hash / validators / segmenter) use the resolved local path.
458         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
459         video_id = compute_video_id(local_video)
460         was_reingest = db_instance.get_video(video_id) is not None
461
462         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
463         notify("validating")
464         logger.info(f"Running validations for {req.video_path}...")
465         val_results, val_context = registry.run_all_with_context(local_video, global_config)
466         failures = [r for r in val_results if not r.passed]
467
468         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
469         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
470         seg_name = req.segmenter or default_seg
471         segmenter_actual = None
472         segmentation_ran = False
473         response: Optional[Dict[str, Any]] = None
474         try:
475             if failures:
476                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
477                 # status; it no longer destroys the video's prior good segments.
478                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
479                 response = {
480                     "video_id": video_id,
481                     "status": "failed",
482                     "message": "Video failed validation checks.",
483                     "results": [r.model_dump() for r in val_results],
484                 }
485                 return response
486
487         # 2. Run segmenter & indexer
488         logger.info("[API] Video passed checks. Segmenting and indexing...")
489         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
490
```

```

491     segmenter_cls = segmenter_registry.get(seg_name)
492     if not segmenter_cls:
493         # Submit-time validation already 400s unknown names; this is the defensive
494         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
495         available = list(segmenter_registry.list_available().keys())
496         response = {
497             "video_id": video_id,
498             "status": "failed",
499             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
500             "results": [r.model_dump() for r in val_results],
501             "play_segments": [],
502         }
503         return response
504
505     logger.info(f"[API] Using segmenter: {seg_name}")
506     notify(f"segmenting ({seg_name})")
507     segmenter_actual = seg_name
508     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
509     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
510     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
511     segmenter = segmenter_cls(db_instance, global_config)
512     try:
513         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
514     except Exception:
515         # A raising segmenter must not leave half-written rows: restore the pre-run
516         # state (same destroy-after-confirm contract as the Failure branch), then
let
517         # the job worker record the failure.
518         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
519         raise
520     segmentation_ran = True
521
522     if isinstance(result, Failure):
523         logger.error(f"[API] Segmenter failed: {result.message}")
524         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
525         response = {
526             "video_id": video_id,
527             "status": "failed",
528             "message": f"Segmenter '{seg_name}' failed: {result.message}",
529             "results": [r.model_dump() for r in val_results],
530             "play_segments": [],
531         }
532         return response
533
534     segments = result.value if hasattr(result, "value") else result
535     notify("finalizing")
536     _finalize_reingest(video_id, segments, play_wm, cand_wm)
537
538     response = {
539         "video_id": video_id,
540         "status": "success",
541         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
542         "results": [r.model_dump() for r in val_results],
543         "play_segments": segments,
544     }
545     return response
546 finally:
547     # Always emit the per-video observability report (next to the video, or to
548     # report.output_dir). Never raises. Surface the paths in the response.

```

```

549         paths = emit_indexer_report(
550             db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
551             val_results=val_results, val_context=val_context,
552             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
553             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
554         )
555         if paths and isinstance(response, dict):
556             response["reports"] = paths
557
558
559     @app.post("/api/process", status_code=202)
560     def process_video(req: ProcessRequest, request: Request):
561         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
562
563         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
564         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
565         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
566         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
567         """
568         # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
569         # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
570         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
571         try:
572             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
573         except edge_auth.EdgeAuthError as e:
574             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
575
576         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
577         try:
578             validate_input_path(req.video_path, allowed_root=allowed_root)
579             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
580             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
581             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
582         except ValueError as e:
583             raise HTTPException(status_code=400, detail=str(e)) from e
584         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
585         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
586         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
587         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
588         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
589         # rejects a non-local URI as out-of-root.
590         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
591             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
592         if req.segmenter and not segmenter_registry.get(req.segmenter):
593             available = list(segmenter_registry.list_available().keys())
594             raise HTTPException(
595                 status_code=400,
596                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
597     )
598
599     job_id = uuid.uuid4().hex

```

```

600         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
601                                     req.player_pool, req.max_frames, owner_id=owner_id,
602                                     locale=req.locale):
603             raise HTTPException(status_code=500, detail="Failed to persist job record.")
604         _get_executor().submit(_drain_due_jobs)
605         return JSONResponse(
606             status_code=202,
607             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
608         )
609
610
611     @app.get("/api/jobs/{job_id}/cost")
612     def get_job_cost(job_id: str):
613         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
614         truth (matches
615         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
616         ``cost_usd`` is
617         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
618         cost = db_instance.get_job_cost(job_id)
619         if cost is None:
620             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
621         fx = global_config.billing.fx_inr_per_usd
622         usd = cost.get("cost_usd")
623         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
624         cost["fx_inr_per_usd"] = fx
625         return cost
626
627
628     @app.get("/api/videos/{video_id}/cost")
629     def get_video_cost(video_id: str):
630         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
631         ``total_cost_inr`` + the
632         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
633         several."""
634         agg = db_instance.get_video_cost(video_id)
635         fx = global_config.billing.fx_inr_per_usd
636         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
637         agg["fx_inr_per_usd"] = fx
638         return agg
639
640
641     @app.get("/api/jobs/{job_id}")
642     def get_job(job_id: str):
643         """Job state: {status, progress, result, reports}. ``status`` = execution state
644         (queued|running|done|failed); the pipeline verdict is result["status"]."""
645         job = db_instance.get_job(job_id)
646         if not job:
647             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
648         result = job.get("result")
649         return {
650             "job_id": job["id"],
651             "status": job["status"],
652             "progress": job.get("progress"),
653             "video_id": job.get("video_id"),
654             "error": job.get("error"),
655             "result": result,
656             "reports": (result or {}).get("reports"),
657             "created_at": job.get("created_at"),
658             "started_at": job.get("started_at"),
659             "finished_at": job.get("finished_at"),
660         }
661
662
663     # --- Chunked upload (B2, PR-2) -----
664     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
665     in

```



```

660 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
661 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
662 # sequential-part logic, and abort cleanup ? is unchanged.
663 from backend.api import uploads as uploads_api
664
665 app.include_router(uploads_api.router)
666
667
668 # --- Highlight download (B3, PR-2) -----
669
670 @app.get("/api/highlights/{video_id}/{filename}")
671 def download_highlight(video_id: str, filename: str):
672     """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
673     (which only writes into output_dir; the browser previously got back an unreachable
674     server-local path)."""
675     try:
676         name = safe_output_filename(filename)
677     except ValueError as e:
678         raise HTTPException(status_code=400, detail=str(e)) from e
679     if not db_instance.get_video(video_id):
680         raise HTTPException(status_code=404, detail="Indexed video record not found.")
681     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
682     path = os.path.join(output_dir, name)
683     try:
684         path = resolve_under_root(path, output_dir)
685     except ValueError as e:
686         raise HTTPException(status_code=400, detail=str(e)) from e
687     if not os.path.isfile(path):
688         raise HTTPException(status_code=404,
689                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
690     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
691     return FileResponse(path, media_type=media, filename=name)
692
693
694 @app.post("/api/query")
695 def query_play_segments(req: QueryRequest):
696     filters = {
697         "server": req.server,
698         "receiver": req.receiver,
699         "duration_min": req.duration_min,
700         "event_type": req.event_type
701     }
702     segments = db_instance.query_play_segments(req.video_id, filters)
703     return {"play_segments": segments}
704
705 @app.post("/api/compile")
706 def compile_highlights(req: CompileRequest):
707     video = db_instance.get_video(req.video_id)
708     if not video:
709         raise HTTPException(status_code=404, detail="Indexed video record not found.")
710
711     # Retrieve matching play segments from db
712     all_segments = db_instance.query_play_segments(req.video_id, {})
713     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
714
715     if not matched_segments:
716         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717
718     # Reject path traversal / absolute output names (os.path.join would otherwise let an
719     # absolute or '..'-laden output_filename write anywhere on disk).

```

```

720     try:
721         out_name = safe_output_filename(req.output_filename)
722     except ValueError as e:
723         raise HTTPException(status_code=400, detail=str(e)) from e
724
725     # The configured shot-count floor (stitching.min_shot_count) applies on the API
726     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
727     # metadata are exempt: the floor filters known counts only, so explicitly
728     # requested manual/legacy segments can't silently vanish under the default floor.
729     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
730     kept = []
731     for s in matched_segments:
732         meta = s.get("metadata") or {}
733         sc = meta.get("shot_count") if isinstance(meta, dict) else None
734         try:
735             if sc is not None and int(sc) < stitching.min_shot_count:
736                 continue
737             except (TypeError, ValueError):
738                 pass # unparseable count -> treat as unknown, keep
739             kept.append(s)
740     if len(kept) < len(matched_segments):
741         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
742                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
743         if not kept:
744             raise HTTPException(status_code=400,
745                                 detail=f"All {len(matched_segments)} matching segments fall
below "
746                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
747
748     # Extract start/end time pairs
749     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
750
751     # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
752     # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
753     # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
754     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
755     localized_watermark = localize_watermark(stitching.watermark, req.locale)
756     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
757                               end_padding=stitching.end_padding,
watermark=localized_watermark)
758     try:
759         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
760         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
761         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
762         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
763         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
764         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
765         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
766         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
767         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md

```

```

§4.3).
768         download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769         return {"status": "success", "filepath": out_path, "download_url": download_url}
770     except Exception as e:
771         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
772
773     # --- CLI Debug Utility & Orchestration ---
774     def run_cli_diagnose_clip(video_path: str, timestamp: float):
775         """CLI debugging utility to examine segment properties around a timestamp."""
776         print("-" * 60)
777         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
778         print("-" * 60)
779
780         cfg = load_config() # legacy wrapper still works, returns dict for now
781
782         # 1. Validation check diagnostics
783         print("[DIAGNOSTIC] Running validation framework...")
784         results = registry.run_all(video_path, cfg)
785         for r in results:
786             status_symbol = "[PASS]" if r.passed else "[FAIL]"
787             print(f"    {status_symbol} {r.validator_name}: {r.message}")
788             if r.details:
789                 print(f"        Details: {r.details}")
790
791         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
792         print("-" * 60)
793         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
794         import cv2
795         cap = cv2.VideoCapture(video_path)
796         if not cap.isOpened():
797             print("[FAIL] OpenCV could not open video.")
798             return
799
800         fps = cap.get(cv2.CAP_PROP_FPS)
801         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
802
803         start_frame = max(0, int((timestamp - 3.0) * fps))
804         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
805
806         # Measure frame-by-frame changes in sample region
807         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
808         prev_gray = None
809         motions = []
810
811         for _ in range(start_frame, end_frame):
812             ret, frame = cap.read()
813             if not ret:
814                 break
815             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
816             gray = cv2.GaussianBlur(gray, (21, 21), 0)
817
818             if prev_gray is not None:
819                 diff = cv2.absdiff(prev_gray, gray)
820                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
821                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
822                 motions.append(motion_ratio)
823                 prev_gray = gray
824
825         cap.release()
826
827         if motions:
828             avg_motion = sum(motions) / len(motions)
829             # Support both legacy dict (from wrapper) and future direct model

```

```

830         if hasattr(cfg, "indexing"):
831             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
832         else:
833             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
834         print(f" Sample Frame Count: {len(motions)}")
835         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
836         if avg_motion > threshold:
837             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
838         else:
839             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")

```

36. assistant (2026-06-22T09:08:57.936Z)

Now let me check the worker entry point for the CLI command-line usage:

37. user (2026-06-22T09:09:00.117Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/worker/__main__.py

38. user (2026-06-22T09:09:00.820Z)

```

1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv
20     import json
21     import logging
22     import os
23     import sys
24     import tempfile
25     from typing import Any, List
26
27     from backend import storage
28     from backend.compute import ComputeJobSpec, get_compute_backend
29     from backend.config import (
30         StartupCheckError,
31         enforce_startup_checks,
32         load_config,
33         probe_cuda_available,
34     )

```

```

35     from backend.infrastructure.database import Database
36     from backend.pipeline.segmenters.base import segmenter_registry
37     from backend.pipeline.validators.base import registry as validator_registry
38     from backend.reporting import emit_indexer_report
39     from backend.results import Failure
40     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
41     from backend.utils.hashing import compute_video_id
42
43     logger = logging.getLogger("backend.worker")
44
45
46     def _coerce(v: str) -> Any:
47         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
48         str."""
49         low = v.lower()
50         if low in ("true", "false"):
51             return low == "true"
52         for cast in (int, float):
53             try:
54                 return cast(v)
55             except ValueError:
56                 pass
57         return v
58
59     def _apply_overrides(config: Any, sets: List[str]) -> None:
60         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
61         indexing.skip_ai_handoff=true)."""
62         for kv in sets or []:
63             key, sep, val = kv.partition("=")
64             if not sep:
65                 raise ValueError(f"--set expects k=v, got {kv!r}")
66             parts = key.split(".")
67             obj = config
68             for p in parts[:-1]:
69                 obj = getattr(obj, p)
70             setattr(obj, parts[-1], _coerce(val))
71
72     def _write_intervals_csv(segments: List[dict], path: str) -> None:
73         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
74         friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
88     None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{
95                     "start": float(s.get("start_time", 0.0)),
96                     "end": float(s.get("end_time", 0.0))
97                 } for s in segments],
98             }, f, indent=2)

```

```

96
97
98     def _run_startup_checks(config: Any) -> bool:
99         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
100             best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
101             cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
102             (returns True, ZERO work) when no deployment.profile is selected.
103
104             The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
105             torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
106             WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
107             no-op of work, not just of output."""
108         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
109         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
110         try:
111             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
112             return True
113         except StartupCheckError:
114             return False # already logged at ERROR by enforce_startup_checks
115
116
117     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
118         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
119             outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
120             container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
121
122             A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
123             poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
124             timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
125         from backend.infrastructure.execution_policy import PolicyTimeout
126
127         spec = ComputeJobSpec(
128             video_uri=args.video_uri, out_uri=args.out_uri,
129             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
130         )
131         try:
132             result = compute.run_infer(spec)
133         except PolicyTimeout as e:
134             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
135             return 4
136         logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
137                     result.video_id, result.returncode, result.outputs_uri)
138         return result.returncode
139
140
141     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
142                           status: str, storage_cfg: dict, scratch: str) -> None:
143         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
144             dispatcher's bucket-poll. Best-effort ? never raises into the worker's success

```

```

return."""
145         try:
146             marker = {"video_id": video_id, "returncode": returncode,
147                       "segment_count": segment_count, "status": status}
148             local = os.path.join(scratch, "_done.json")
149             with open(local, "w", encoding="utf-8") as f:
150                 json.dump(marker, f)
151             storage.put(local, done_uri, config=storage_cfg)
152             logger.info("[worker] wrote completion marker -> %s", done_uri)
153         except Exception as e: # never fail a good run because the marker push hiccuped
154             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
155
156
157     def cmd_infer(args: argparse.Namespace) -> int:
158         config = load_config(args.config) if args.config else load_config()
159         _apply_overrides(config, args.set)
160         if not _run_startup_checks(config):
161             return 2
162
163         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
164         # process.
165         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
166         # through to the
167         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
168         # (the
169         # dispatcher forces compute_target=local_gpu), so it never recursively re-
170         # dispatches.
171         compute = get_compute_backend(config)
172         if compute is not None:
173             return _dispatch_remote(compute, args)
174
175         storage_cfg = config.storage.model_dump()
176
177         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
178         os.makedirs(scratch, exist_ok=True)
179         config.output.output_dir = scratch
180
181         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
182         # to scratch.
183         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
184                               config=storage_cfg)
185         print(f"[worker] pulled {args.video_uri} -> {local_video}")
186
187         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
188         # byte-identical).
189         video_id = compute_video_id(local_video)
190
191         # 3. VALIDATE (same registry as the main CLI).
192         val_results, val_context = validator_registry.run_all_with_context(local_video,
193                               config)
194         failures = [r for r in val_results if not r.passed]
195         db = Database(os.path.join(scratch, config.output.db_name))
196         db.add_video(video_id, local_video, "failed" if failures else "processed",
197                     [r.model_dump() for r in val_results])
198
199         segments: List[dict] = []
200         segmenter_actual = None
201         segmentation_ran = False
202         status = "failed"
203         try:
204             if failures:
205                 print("[worker] validation FAILED: "
206                       + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
207                 return 2
208             seg_name = args.segmenter or config.indexing.default_segmenter

```

39. assistant (2026-06-22T09:09:04.024Z)

Perfect! Now I have enough context. Let me create the structured output with all the required information about the configured/checkpointed state and the nightly regression design:

40. user (2026-06-22T09:09:41.523Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 5967 bytes): {"summary": "The badminton-highlight-indexer defines a CONFIGURED + CHECKPOINTED pipeline state for nightly regression testing. The system uses a typed Pydantic AppConfig model (backend/config/models.
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

41. assistant (2026-06-22T09:09:43.901Z)

Let me call the tool correctly with properly escaped JSON:

42. user (2026-06-22T09:10:42.300Z)

Structured output provided successfully

43. assistant (2026-06-22T09:11:19.661Z)

Perfect. The structured output has been returned. Let me now provide a final summary in prose for clarity:

Summary: Configured + Checkpointed Pipeline State & Nightly Regression Design

I have thoroughly mapped the badminton-highlight-indexer codebase to define exactly what "running as configured and checkpointed" means and how the nightly regression operates.

****CONFIGURED + CHECKPOINTED STATE****

****1. Typed Configuration (source of truth)****

- **Root model:** ``backend/config/models.py::AppConfig`` (Pydantic BaseModel)
- **Precedence chain (low ? high):**
 - 1. Built-in ``AppConfig()`` defaults in `models.py`
 - 2. ``config.json`` file (at ``Path("config.json")`` CWD-relative, or ``RALLY_APP_HOME/config.json`` if env set)
 - 3. Deployment profile preset (``backend/config/presets.py``: `truly-local / cloud-serving / cpu-mock`)
 - 4. Environment variable overrides at consumption sites (e.g., ``WASB_WEIGHTS``, ``DEPLOYMENT_PROFILE``)
- **Config file resolution:** ``backend/utils/app_home.py::default_config_path()`` ? returns ``(RALLY_APP_HOME or ".") / "config.json"``; unset env means CWD-relative (byte-identical to pre-P0a behavior)
- **Loader:** ``backend/config/loader.py::load_config(path=None)`` handles deep-merge of preset + file overrides

****2. WASB Shuttle-Trajectory Checkpoint****

- **Weights path:** ``config.json`` line 61: ``indexing.wasb.weights_path = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"``
- **Resolution:** ``backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg()`` reads the config value
- **Native runner override (Linux GPU box):** env var ``WASB_WEIGHTS`` takes precedence over config
- **Safety:** The nightly regression SKIPS re-extraction (uses cached trajectory CSVs), so checkpoint updates are invisible to the tripwire


```

**3. Default Segmenter**
- **Config value:** `config.json` line 26: `"default_segmenter": "gemini"`
- **Built-in default:** `backend/config/models.py::IndexingConfig.default_segmenter = "yolo_hybrid"`
- **Resolution:** `backend/main.py` line 469: `seg_name = req.segmenter or default_seg = getattr(config.indexing, "default_segmenter", "motion")`

**4. PIN FOR REPRODUCIBILITY**
- Commit a locked `eval_baselines/nightly_config.json` (snapshot of production config.json)
- Commit `eval_baselines/nightly_baseline.json` (frozen metric snapshot)
- Store locally (owner's machine, not repo): golden corpus manifest + trajectory CSVs + features JSONs (licensed/minors-free, non-extracted)
- A config change (intentional baseline refresh) triggers exit code 4 (STALE), signaling manual `--update-baseline` snapshot

```

****HEADLESS RECIPE (to run the configured pipeline on a video)****

```

` `` `bash

```

Resolve app-home and load config

```

python -c "
from backend.config import load_config
from backend.pipeline.segmenters.base import segmenter_registry
from backend.infrastructure.database import Database

cfg = load_config() # CWD-relative config.json or RALLY_APP_HOME/config.json

```

Instantiate segmenter (uses indexing.default_segmenter from config)

```

seg_name = cfg.indexing.default_segmenter # e.g., 'gemini'
segmenter_cls = segmenter_registry.get(seg_name)
db = Database('output/sports_indexer.db')
segmenter = segmenter_cls(db, cfg)

```

Run on a video

```

video_id = compute_video_id('path/to/test.mp4')
result = segmenter.process_video(video_id, 'path/to/test.mp4')
segments = result.value if hasattr(result, 'value') else result

```

Segments = List[{start_time, end_time, shot_count, ...}]

```

print(f'Detected {len(segments)} rallies')
"
` `` `

```

Or via the worker (handles storage URIs, cached trajectories, overrides):

```

` `` `bash
python -m backend.worker infer \
  --video-uri "path/to/test.mp4" \
  --out-uri output/scratch \
  --segmenter wasb_hybrid \
  --set "indexing.skip_ai_handoff=true" # fully local, no Gemini
` `` `

```

****NIGHTLY REGRESSION DESIGN****

****Purpose:**** Tripwire that re-scores a local golden corpus against CACHED trajectories (no GPU re-extraction) and compares windowing metrics to a frozen baseline.

****Entry point:**** `python scripts/nightly\_regression.py [--update-baseline]`

****What it does:****

1. Loads golden corpus from local manifest (`output/human\_lovo\_manifest.json`)
2. Scores 3 tracked windowing configs:
 - ****DEFAULT:**** Production-shipped (now = R1 cap=6 + R4 inactivity trim after #204 flip)

- **MILESTONE:** Explicit R1 config (cap=6 s + 1.5 s inactivity trim)
- **BASELINE_OFF:** Pre-R1-flip floor (all windowing knobs OFF)

3. Compares aggregate metrics (tolF1, f1@0.5, merge_rate) + per-video over-segmentation guards to `eval\_baselines/nightly\_baseline.json`

4. Exit codes:

- **0:** PASS (metrics stable within tolerance)
- **1:** REGRESSION (a metric dropped beyond threshold OR per-video split/merge count increased)
- **4:** STALE (corpus changed OR config definition drifted)
- **2:** Error (manifest/corpus missing)

Snapshot baseline (after intentional improvement):

```
```bash
```

```
python scripts/nightly_regression.py --update-baseline
```

**Updates eval\_baselines/nightly\_baseline.json with current scores**

**Commit it to lock the new floor**

```
```
```

REEL COUNT ASSERTION (for short golden videos)

The nightly currently gates on rally *detection quality* (F1, precision, recall, windowing metrics). For a simple reel-count regression:

1. **Count rallies per video:** Query `backend.eval.rally\_seg\_eval.evaluate()` output (per-video "num_pred" field)
2. **Store expected counts:** In `eval\_baselines/fixtures/<fixture>/expected.json` (structure: see `backend.eval.golden\_fixtures.py`)
3. **Assert in CI:** `assert current\_reel\_count == baseline\_reel\_count` per golden fixture

Example: 2?3 short golden videos (synthetic or real badminton clips, 1?3 min each, minors-free). Once the nightly baseline is established on the owner's GPU box, the counts become invariant (same video + same config + cached trajectories = deterministic reel count).

```
---
```

All paths and code references are absolute to: `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/`